

باسمه تعالی



دانشکده مهندسی کامپیوتر

مبانی هوش مصنوعی و کاربردها

تیم Untitled: فاطمه سلطانی - حسنا رضایتمند

استاد: دکتر حسین کارشناس

دستیاران آموزشی:

مهدی مهدیه

یونس ایوبی راد

دانیال شفیعی

پویا اسفندانی

1- جستجوی مسیر بهینه در محیط با دشمن و موانع

هدف سوال

هدف این پروژه یافتن مسیر بهینه برای عامل (Agent) در یک محیط شبکه‌ای بود که شامل موانع، دشمنان و اهداف (Targets) است. الگوریتم‌ها باید مسیری پیدا کنند که عامل را از موقعیت اولیه به تمام اهداف برساند و در عین حال از برخورد با دشمن و موانع جلوگیری کنند و هزینه مسیر را کمینه کنند.

روش کار و الگوریتم‌ها

1. BFS (جستجوی اول-عرضی):

یک الگوریتم جستجوی کامل است که همه حالات ممکن را به ترتیب افزایش عمق بررسی می‌کند. از یک صف استفاده می‌کند و مسیر کوتاه‌ترین تعداد گام‌ها را پیدا می‌کند. محدودیت: BFS فقط تعداد گام‌ها را کمینه می‌کند و هزینه واقعی مسیر (بر اساس نوع زمین و موانع) را در نظر نمی‌گیرد.

2. (UCS) Uniform Cost Search:

جستجوی کامل با هزینه یکنواخت، که مسیر با کمترین هزینه واقعی را پیدا می‌کند. از یک صف اولویت‌دار استفاده می‌کند که مسیرهای با هزینه کمتر اولویت دارند و اطمینان می‌دهد که مسیر بهینه از نظر هزینه (Terrain Cost و برخورد با دشمن) پیدا شود.

ساختار کلی:

در تابع UCS از یک صف اولویت‌دار (Priority Queue) برای نگهداری حالت‌ها استفاده شده است که اولویت آن‌ها بر اساس هزینه تجمعی (cost) تعیین می‌شود. در ابتدا، گره آغازین با هزینه صفر در صف قرار داده می‌شود. دیکشنری `cost_so_far` هزینه‌ی کمینه رسیدن به هر حالت را نگهداری می‌کند. در هر مرحله، گره با کمترین هزینه از صف خارج می‌شود. اگر حالت فعلی یک حالت هدف (`is_goal_state`) باشد، مسیر طی شده برگردانده می‌شود. در غیر این صورت، تمام حالت‌های جانشین با استفاده از:

```
for action, step_cost, next_state in state.get_successors
```

بررسی می‌شوند.

هزینه جدید هر حرکت به صورت زیر محاسبه می‌شود:

`new_cost = cost + step_cost`. اگر حالت جدید قبلاً بازدید نشده یا هزینه‌ی جدید کمتر از هزینه‌ی قبلی باشد، مقدار به‌روز شده در صف قرار می‌گیرد.

الگوریتم تا یافتن هدف یا خالی شدن صف ادامه دارد.

3. DLS و IDS (جستجوی عمق محدود و تکراری):

(DLS) Depth-Limited Search: جستجوی عمق محدود که تا یک حد مشخص عمق جلو می‌رود و از

پنهایت شدن جلوگیری می‌کند.

(IDS) Iterative Deepening Search: اجرای متوالی DLS با افزایش تدریجی عمق تا رسیدن به هدف. مزیت:

حافظه کمتری نسبت به BFS و UCS استفاده می‌کند و همچنان جواب کامل پیدا می‌کند. یک متغیر visited

برای ذخیره حالت‌های قبلی به تابع اضافه شد تا در زمان معقول‌تری بازی اجرا شود.

4. A* Search (جستجوی A*):

یک جستجوی هدایت شده است که از ترکیب **هزینه مسیر طی شده و هزینه تخمینی رسیدن به اهداف**

باقی‌مانده (Heuristic) استفاده می‌کند.

Heuristic: ترکیبی از حداقل هزینه رسیدن به نزدیک‌ترین هدف و هزینه (MST (Minimum Spanning Tree)

بین اهداف و فاصله از دشمن است.

ما از یک تابع اکتشافی **سازگار (Admissible)** و **آگاهانه (Informed)** برای الگوریتم A* استفاده کردیم که به طور قابل

توجهی کارایی جستجو را در مقایسه با اکتشافی ساده فاصله منهن، افزایش می‌دهد. این تابع بر اساس رویکرد دو بخشی

زیر عمل می‌کند تا یک کران پایین قوی برای هزینه باقی‌مانده ارائه دهد:

1. **هزینه آگاه به زمین (C terrain):** ابتدا، با استفاده از الگوریتم دایجسترا، کمترین هزینه واقعی عبور از زمین

(شامل هزینه‌های چمن: ۵ و بوته: ۱۰۰) بین تمام نقاط شبکه را به صورت پیش‌محاسبه (Pre-computed) ذخیره

می‌کنیم تا از کم‌تخمینی ناشی از نادیده گرفتن هزینه‌های بالا جلوگیری شود.

2. **درخت پوشای مینیمم (MST):** تابع ارزیابی $h(n)$ برابر است با:

$$h(n) = C_{\text{terrain}}(\text{Agent} \rightarrow \text{Nearest Target}) + \text{Cost of MST}(\text{Remaining Targets})$$

هزینه MST، با استفاده از الگوریتم پریم و بر اساس C terrain، محاسبه می‌شود و یک کران پایین دقیق برای

حداقل هزینه لازم جهت اتصال و جمع‌آوری تمام اهداف باقیمانده فراهم می‌کند. این رویکرد تضمین می‌کند

که A* بهینه‌ترین مسیر را با حداقل نودهای گسترش یافته پیدا کند. همچنین فاصله عامل از دشمن محاسبه

شده و به هیوریستیک اضافه می‌شود. با دادن وزن بیشتر به تابع هیوریستیک توانستیم با تعداد کمتری

expanded node به جواب برسیم.

تحلیل و مقایسه:

- IDS و BFS حافظه بیشتری مصرف می کنند ولی مسیر کوتاه به لحاظ تعداد حرکت را پیدا می کنند.
- UCS و *A مسیر کم هزینه واقعی را پیدا می کنند که هزینه عبور از زمین و اجتناب از دشمنان را لحاظ می کند.
- *A با استفاده از Heuristic مناسب، سریعتر از UCS عمل می کند و تعداد حالت های بررسی شده کاهش می یابد.

2- پیش‌بینی قطر سیارک‌ها با رگرسیون خطی و الگوریتم SGD

هدف سؤال

هدف این بخش از پروژه، طراحی و پیاده‌سازی یک مدل رگرسیون خطی برای پیش‌بینی قطر سیارک‌ها بر اساس ویژگی‌های فیزیکی و مداری موجود در داده‌های سازمان ناسا (JPL Dataset) بوده است. در این پروژه الگوریتم کاهش گرادینت تصادفی (Stochastic Gradient Descent – SGD) به صورت کامل از پایه (From Scratch) پیاده‌سازی شد تا فرآیند یادگیری ضرایب مدل بدون استفاده از کتابخانه‌های آماده انجام گیرد. این مدل با هدف درک مفاهیم بهینه‌سازی، کاهش خطا (MSE) و تحلیل روند همگرایی طراحی شده است.

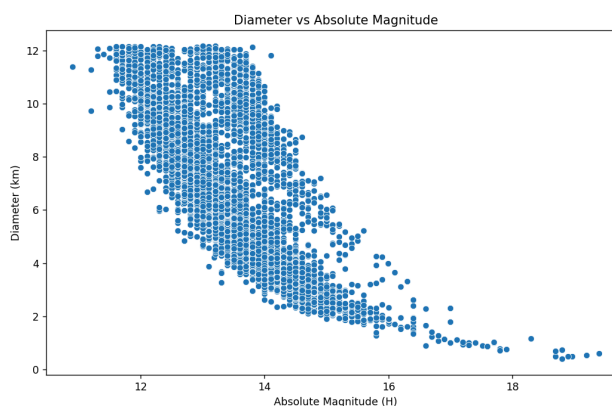
کد و الگوریتم‌ها

1. آماده‌سازی داده‌ها:

در ابتدا داده‌های مربوط به ویژگی‌های فیزیکی و مداری سیارک‌ها از فایل NASA_JPL_Dataset.csv بارگذاری شدند. داده‌های پرت با مقدار RMS بیشتر از ۱ حذف و ستون‌های غیرضروری مانند شناسه‌ها و پارامترهای با خطای زیاد حذف شدند. به منظور نرمال‌سازی و بهبود توزیع هدف، از لگاریتم قطر ($\log_diameter$) به جای قطر واقعی استفاده شد. سپس داده‌ها به مجموعه‌های آموزش (Train) و آزمون (Test) تقسیم، داده‌های متنی کدگذاری، و مقادیر عددی با StandardScaler مقیاس‌بندی شدند.

2. تحلیل داده‌ها (EDA):

با رسم هیستوگرام و ماتریس همبستگی، رابطه‌ی بین ویژگی‌ها بررسی شد. مشخص گردید که بین قدر مطلق (H) و قطر سیارک رابطه‌ی معکوس وجود دارد؛ هرچه H بیشتر باشد، قطر سیارک کوچک‌تر است.



3. پیاده‌سازی مدل از پایه:

کلاس SGDRegressorScratch برای اجرای الگوریتم SGD طراحی شد. این کلاس شامل مراحل زیر است:

- **__init__**: تنظیم پارامترهای مدل مانند نرخ یادگیری (learning_rate)، تعداد تکرار (n_epochs)، اندازه‌ی دسته (batch_size)، مومنتوم (beta) و کاهش نرخ یادگیری (lr_decay).
- **augment_x**: افزودن ستون بایاس برای مدل رگرسیون.
- **fit**: بخش اصلی آموزش که شامل:

1. مقداردهی اولیه‌ی وزن‌ها به صورت تصادفی
2. تقسیم داده‌ها به batch های کوچک
3. محاسبه پیش‌بینی، خطا و گرادیان MSE برای هر batch
4. به‌روزرسانی وزن‌ها بر اساس فرمول مومنتوم:
5. ثبت مقدار خطا (MSE) در هر epoch و کاهش تدریجی نرخ یادگیری.

- For each batch:
 - Compute predictions:

$$\hat{y} = X_{batch} \cdot w$$

- Compute error:

$$e = \hat{y} - y_{batch}$$

- Compute the gradient of the Mean Squared Error (MSE):

$$\nabla = \frac{2}{m} X_{batch}^T e$$

- Update the velocity (momentum term):

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla$$

- Update the weights:

$$w := w - \eta_t \cdot v_t$$

- Adjust the learning rate using decay:

$$\eta_t = \frac{\eta_0}{1 + \text{lr_decay} \times \text{epoch}}$$

- **plot_loss_curve**: رسم نمودار تغییر خطا در طول آموزش (Loss Curve).
- **predict**: پیش‌بینی خروجی بر اساس وزن‌های آموزش‌دیده.

4. تنظیمات مدل:

- تعداد تکرار: Epochs = 200
 - نرخ یادگیری اولیه: 0.005
 - اندازه Batch: 2000
 - مومنتوم: $\beta = 0.9$
 - کاهش نرخ یادگیری: lr_decay = 0.001
5. مدل در هر epoch وزن‌ها را به‌روزرسانی کرده و مقدار MSE را چاپ می‌کند تا روند کاهش خطا بررسی شود.

نتایج نهایی

در پایان آموزش، مدل با موفقیت به همگرایی رسید و مقدار خطا در هر epoch به طور پیوسته کاهش یافت. مقادیر نهایی در پایان آموزش به صورت زیر گزارش شدند:

Epoch 197/200 - MSE: 0.018265

Epoch 198/200 - MSE: 0.018258

Epoch 199/200 - MSE: 0.018250

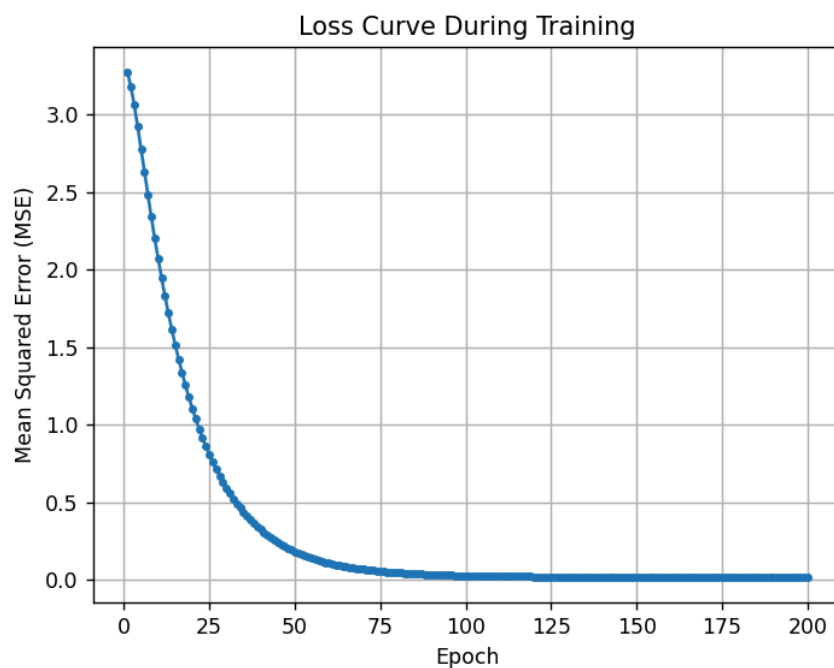
Epoch 200/200 - MSE: 0.018243

R^2 Scratch SGD Score: 0.9121

MAE SGD Score: 0.1026

MSE SGD Score: 0.0181

- مقدار $R^2 = 0.91$ نشان‌دهنده دقت بالای مدل و توانایی خوب در پیش‌بینی قطر سیارک‌هاست.
- خطای MSE پایین و $MAE \approx 0.10$ حاکی از همگرایی مناسب الگوریتم و دقت قابل قبول پیش‌بینی است.
- نمودار خطا نیز روندی نزولی داشت که تأیید می‌کند مدل به تدریج به حداقل خطا رسیده است.



3- جستجوی مسیر بهینه در محیط با دشمن و موانع

هدف سوال

هدف این پروژه یافتن مسیر بهینه برای عامل (Agent) در یک محیط شبکه‌ای بود که شامل موانع، دشمنان و اهداف (Targets) است. الگوریتم‌ها باید مسیری پیدا کنند که عامل را از موقعیت اولیه به تمام اهداف برساند و در عین حال از برخورد با دشمن و موانع جلوگیری کنند و هزینه مسیر را کمینه کنند.

روش کار و الگوریتم‌ها

1. BFS (جستجوی اول-عرضی):

یک الگوریتم جستجوی کامل است که همه حالات ممکن را به ترتیب افزایش عمق بررسی می‌کند. از یک صف استفاده می‌کند و مسیر کوتاه‌ترین تعداد گام‌ها را پیدا می‌کند. محدودیت: BFS فقط تعداد گام‌ها را کمینه می‌کند و هزینه واقعی مسیر (بر اساس نوع زمین و موانع) را در نظر نمی‌گیرد.

2. (UCS) Uniform Cost Search:

جستجوی کامل با هزینه یکنواخت، که مسیر با کمترین هزینه واقعی را پیدا می‌کند. از یک صف اولویت‌دار استفاده می‌کند که مسیرهای با هزینه کمتر اولویت دارند و اطمینان می‌دهد که مسیر بهینه از نظر هزینه (Terrain Cost و برخورد با دشمن) پیدا شود.

ساختار کلی:

در تابع UCS از یک صف اولویت‌دار (Priority Queue) برای نگهداری حالت‌ها استفاده شده است که اولویت آن‌ها بر اساس هزینه تجمعی (cost) تعیین می‌شود. در ابتدا، گره آغازین با هزینه صفر در صف قرار داده می‌شود. دیکشنری `cost_so_far` هزینه‌ی کمینه رسیدن به هر حالت را نگهداری می‌کند. در هر مرحله، گره با کمترین هزینه از صف خارج می‌شود. اگر حالت فعلی یک حالت هدف (`is_goal_state`) باشد، مسیر طی‌شده برگردانده می‌شود. در غیر این صورت، تمام حالت‌های جانشین با استفاده از:

```
for action, step_cost, next_state in state.get_successors
```

بررسی می‌شوند.

هزینه جدید هر حرکت به صورت زیر محاسبه می‌شود:

$new_cost = cost + step_cost$. اگر حالت جدید قبلاً بازدید نشده یا هزینه‌ی جدید کمتر از هزینه‌ی قبلی باشد، مقدار به‌روز شده در صف قرار می‌گیرد.

الگوریتم تا یافتن هدف یا خالی شدن صف ادامه دارد.

3. DLS و IDS (جستجوی عمق محدود و تکراری):

(DLS) Depth-Limited Search: جستجوی عمق محدود که تا یک حد مشخص عمق جلو می‌رود و از بی‌نهایت شدن جلوگیری می‌کند.

(IDS) Iterative Deepening Search: اجرای متوالی DLS با افزایش تدریجی عمق تا رسیدن به هدف. مزیت: حافظه کمتری نسبت به BFS و UCS استفاده می‌کند و همچنان جواب کامل پیدا می‌کند. یک متغیر visited برای ذخیره حالت‌های قبلی به تابع اضافه شد تا در زمان معقول‌تری بازی اجرا شود.

4. A* Search (جستجوی A*)

یک جستجوی هدایت شده است که از ترکیب **هزینه مسیر طی شده** و **هزینه تخمینی رسیدن به اهداف باقی‌مانده (Heuristic)** استفاده می‌کند.

Heuristic: ترکیبی از حداقل هزینه رسیدن به نزدیک‌ترین هدف و هزینه (MST (Minimum Spanning Tree) بین اهداف و فاصله از دشمن است.

ما از یک تابع اکتشافی **سازگار (Admissible)** و **آگاهانه (Informed)** برای الگوریتم A* استفاده کردیم که به طور قابل توجهی کارایی جستجو را در مقایسه با اکتشافی ساده فاصله منهن، افزایش می‌دهد. این تابع بر اساس رویکرد دو بخشی زیر عمل می‌کند تا یک کران پایین قوی برای هزینه باقی‌مانده ارائه دهد:

1. **هزینه آگاه به زمین (C terrain)**: ابتدا، با استفاده از الگوریتم دایجسترا، کمترین هزینه واقعی عبور از زمین (شامل هزینه‌های چمن: ۵ و بوته: ۱۰۰) بین تمام نقاط شبکه را به صورت پیش‌محاسبه (Pre-computed) ذخیره می‌کنیم تا از کم‌تخمینی ناشی از نادیده گرفتن هزینه‌های بالا جلوگیری شود.
2. **درخت پوشای مینیمم (MST)**: تابع ارزیابی $h(n)$ برابر است با:

$$h(n) = C_{\text{terrain}}(\text{Agent} \rightarrow \text{Nearest Target}) + \text{Cost of MST}(\text{Remaining Targets})$$

هزینه MST، با استفاده از الگوریتم پریم و بر اساس C terrain، محاسبه می‌شود و یک کران پایین دقیق برای حداقل هزینه لازم جهت اتصال و جمع‌آوری تمام اهداف باقیمانده فراهم می‌کند. این رویکرد تضمین می‌کند که A* بهینه‌ترین مسیر را با حداقل نودهای گسترش یافته پیدا کند. همچنین فاصله عامل از دشمن محاسبه

شده و به هیوریستیک اضافه می‌شود. با دادن وزن بیشتر به تابع هیوریستیک توانستیم با تعداد کمتری expanded node به جواب برسیم.

تحلیل و مقایسه:

- BFS و IDS حافظه بیشتری مصرف می‌کنند ولی مسیر کوتاه به لحاظ تعداد حرکت را پیدا می‌کنند.
- UCS و *A مسیر کم‌هزینه واقعی را پیدا می‌کنند که هزینه عبور از زمین و اجتناب از دشمنان را لحاظ می‌کند.
- *A با استفاده از Heuristic مناسب، سریع‌تر از UCS عمل می‌کند و تعداد حالت‌های بررسی شده کاهش می‌یابد.

یافتن مرکز مجموعه‌ای از رشته‌های DNA با استفاده از الگوریتم‌های جستجو

هدف سؤال

هدف این پروژه پیدا کردن مرکز یک مجموعه DNA بود. مرکز یک مجموعه DNA رشته‌ای است که فاصله آن (تعداد جایگاه‌های متفاوت) تا دورترین DNA در مجموعه حداقل باشد. همچنین، مقدار حداقل بیشترین فاصله را به عنوان شعاع (radius) مجموعه تعریف کردیم.

برای این کار هر DNA به صورت یک آرایه کاراکتری و مجموعه DNAها به صورت آرایه دوبعدی مدل شد.

تابع ارزیابی را براساس فاصله بین رشته‌ها تعیین کردیم. برای یک حالت مشخص، بیشترین فاصله تا مجموعه DNAها به عنوان تابع هزینه در نظر گرفته شد.

الگوریتم‌ها

الگوریتم‌های استفاده شده برای جستجوی محلی الگوریتم تپه نوردی (Hill Climbing) و الگوریتم شبیه‌سازی تبرید (Simulated Annealing) هستند.

1- الگوریتم تپه‌نوردی یک روش جستجوی محلی است که همیشه به سمت حالت همسایه با کمترین مقدار تابع ارزیابی حرکت می‌کند.

از یک حالت اولیه (DNA تصادفی) شروع می‌کنیم. همه همسایه‌های حالت جاری تولید می‌شوند. (همسایه حالتی است که فقط در یک موقعیت با حالت جاری متفاوت است). همسایه‌ای که کمترین مقدار بیشترین فاصله (radius) را دارد انتخاب می‌شود و اگر هیچ همسایه‌ای بهتر از حالت فعلی نبود، الگوریتم متوقف می‌شود. این الگوریتم ساده و سریع است اما ممکن است در مینیم‌های محلی گیر کند و جواب بهینه کلی را پیدا نکند.

2- الگوریتم شبیه‌سازی تبرید بهبود یافته Hill Climbing است که می‌تواند گاهی حرکت به حالت بدتر را نیز بپذیرد تا از مینیم‌های محلی فرار کند.

از یک حالت اولیه شروع و دمای اولیه تعیین می‌شود. یک همسایه تصادفی انتخاب می‌شود و تابع ارزیابی آن محاسبه می‌شود. اگر همسایه بهتر باشد، همیشه آن را قبول می‌کنیم. اگر همسایه بدتر باشد، با احتمال زیر قبول می‌شود. دما در هر تکرار کاهش می‌یابد (با نرخ decay برابر alpha). الگوریتم تا رسیدن به حداقل دما یا تعداد تکرار حداکثر ادامه می‌یابد.

این الگوریتم توانایی فرار از مینیم‌های محلی را دارد. پارامتر دما و نرخ کاهش (alpha) روی سرعت مستقیم دارد.

$$P(\text{پذیرش}) = \exp\left(-\frac{\Delta f}{T}\right)$$

مینیم‌های محلی را دارد. پارامتر دما همگرایی و کیفیت جواب نهایی تأثیر

این الگوریتم توانایی فرار از مینیم‌های محلی را دارد. پارامتر دما و نرخ کاهش (alpha) روی سرعت مستقیم دارد.

برای اعتبارسنجی دو الگوریتم پیاده‌سازی شده با استفاده از Brute Force همه حالات ممکن بررسی و بهترین حالت یافت شد و با نتایج بدست آمده از نظر جواب نهایی، شعاع و زمان اجرا مقایسه شد.

```
Simulated Annealing found solution ['c' 'a' 'a' 't' 'c'] with value 2 in 19.925 milliseconds
Hill Climbing found solution ['g' 'a' 'a' 'g' 'c'] with value 2 in 0.000 milliseconds
Brute Force found solution ['c' 'a' 'a' 't' 'c'] with value 2 in 68.377 milliseconds
-----
Simulated Annealing found solution ['a' 'g' 'g' 'g' 'g' 'a' 'c'] with value 2 in 24.358 milliseconds
Hill Climbing found solution ['g' 't' 'g' 'a' 'g' 'g' 'a'] with value 5 in 4.732 milliseconds
Brute Force found solution ['a' 'g' 'g' 'g' 'g' 'a' 'c'] with value 2 in 986.426 milliseconds
-----
Simulated Annealing found solution ['c' 't' 'a' 'c' 't' 'a' 'c' 'c' 't'] with value 3 in 15.625 milliseconds
Hill Climbing found solution ['c' 't' 'a' 'c' 't' 'c' 'a' 'c' 'a'] with value 3 in 0.000 milliseconds
Brute Force found solution ['c' 'c' 'a' 'c' 't' 'c' 'c' 'c' 't'] with value 3 in 12721.718 milliseconds
```

هر دو الگوریتم جستجوی هوشمند تقریباً به جواب درست نزدیک شدند. Simulated Annealing با مقدار مناسب alpha توانست بهتر از Hill Climbing از مینیم‌های محلی خارج شود. زمان اجرای Brute Force بسیار بیشتر بود، اما برای مجموعه‌های کوچک مرجع خوبی برای اعتبارسنجی بود.

